

实验六：动态规划法（二）

对应课程标 3

班级：7 班

姓名：卢瑶

学号：20211018341

一、实验源代码、伪代码、结果和复杂度分析

第四关：求最长单调递增子序列长度。给定一个长度为 n 的数组，找出一个最长的单调递增子序列（不一定连续，但是顺序不能乱）。例如：给定一个长度为 9 的序列 {5, 6, 7, 1, 2, 8, 9}，则其最长的单调递增子序列为 {5, 6, 7, 8, 9}，长度为 5。求序列 {3, 1, 8, 7, 1, 4, 1, 0, 1, 2, 2, 3, 4, 1, 1, 6, 2, 4} 的最长单调递增子序列的长度。

伪码描述：

```
int main() {
    int n;

    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    Length[0] = 1; //递增序列为自身
    for(int i = 1; i < n; i++) {
        int maxl = a[i];
        for(int j = i - 1; j >= 0; j--) {
            寻找0到i-1索引上大于a[i]的元素；
            memory[j]相应位置上记录该元素；
        }
    }
    int max = Length[0], index = 0;
    for(int i = 1; i < sizeof(memory); i++) {
        寻找memory[]中记录子序列最大的元素(memory中全部是小于当前元素的
        数，即满足递增序列的条件);
    }
}
```

```

        Length[i] = max + 1;

        //array 数组重新初始化
        for(int i = 0; i < sizeof(memory); i++)
            memory[i] = 0;
    }
    int max_Len = 0;
    for(int i = 0; i < n; i++) {
        寻找最大值max_Len;
    }
    printf("%d\n", max_Len);

    return 0;
}

```

源代码:

```

#include <stdio.h>
#define max(x, y) x > y ? x : y

int a[100];
int Length[100]; //默认初始化为0
int memory[100];

//给定一个长度为n的数组，找出一个最长的单调递增子序列（不一定连续，但是顺序不能乱）。例如：给定一个长度为7的数组A：5, 6, 7, 1, 2, 8, 9，则其最长的单调递增子序列为5, 6, 7, 8, 9，长度为5。
//example:
// 10
// 3 18 7 14 10 12 23 41 16 24
//状态方程：  $a[j] = \max(Length[j - 1]) + 1$ 

int main() {
    int n;

    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    Length[0] = 1; //递增序列为自身
    for(int i = 1; i < n; i++) {

```

```

    int max1 = a[i];
    // printf("%d\n", a[i]);
    for(int j = i - 1; j >= 0; j--) { //从 0 -> i - 1 的最大值, j 为其相应
的 index
        if(a[j] < max1 ) {
            // printf("%d ", a[j]);
            memory[j] = a[j];
        }
    }
    // printf("\n");
    // for(int i = 0; i < 10; i++){
    //     printf("%d ", memory[i]);
    // }
    // 存储完毕, 对 k 中求最大值,
    int max = Length[0], index = 0; //注意在 循环内 or 循环外 更新 !
    //放循环内的话, 相当于两个数的比较了, 注意细节!
    for(int i = 1; i < sizeof(memory); i++){
        if(memory[i] > 0 && Length[i] > max ){
            max = Length[i];
            index = i;
        }
    }
    //找到前面的数组相对于该元素的 最大值 max 及其 index
    // printf("max is : %d, index is :%d\n", max, index);
    // //前面的最小值 < 当前元素
    Length[i] = max + 1; //更新~
    // printf("\n");
    //array 进行更新 !!
    for(int i = 0; i < sizeof(memory); i++){
        memory[i] = 0;
    }
    int max_Len = 0;
    for(int i = 0; i < n; i++){
        if(Length[i] > max_Len) {
            max_Len = Length[i];
            // printf("%d ", Length[i]);
        }
    }
    printf("%d\n", max_Len);

    return 0;
}

```

运行结果:

10

3 18 7 14 10 12 23 41 16 24

6

时间复杂度分析:

$O(n^2)$

空间复杂度分析:

$O(n)$

第 5 关:矩阵连乘问题,即通过给矩阵连乘时加括号,使得总的矩阵连乘计算量最小。在计算矩阵连乘积时,加括号的方式对计算量有影响。设有 3 个矩阵 A_1, A_2, A_3 连乘,它们的维数分别为 $10 \times 100, 100 \times 5, 5 \times 50$

若按照 $(A_1A_2)A_3$ 方式计算,需要的数乘次数为 $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$

若按照 $A_1(A_2A_3)$ 方式计算,需要的数乘次数为 $50 \times 100 \times 5 + 10 \times 100 \times 50 = 75000$

输入连乘矩阵的个数,每个矩阵的维数。要求输出数乘次数最少时的**加括号方式,及数乘次数**。

伪码描述:

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h> //包含malloc函数
#include <string.h> //包含memset函数
#define min(x, y) x < y ? x : y

/*
example :
6
30 35
35 15
```

```

15 5
5 10
10 20
20 25
*/
//expected: dp[1][6] = 15125
int dp[100][100]; //二维数组存储动态最优解
int p[100][2]; //存储对应Matrix的行列数
int s[10][10];

void traceBack(int i, int j){ //i -> j 的运算顺序（回溯法实现, 树的后序遍历）
    //recursion base
    if(i == j)
        return ;
    int k = s[i][j];
    traceBack(i, k);
    traceBack(k+1, j);
    printf("A%d ... A%d  A%d ... A%d\n", i, k+1, k+1, j);
}

int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);
    for(int i = 1; i <= n; i++){
        for(int j = 0; j < 2; j++){
            scanf("%d", &p[i][j]);
        }
    }
    dp数组初始化，对角线元素为0，其它元素为正无穷。

    for(int L = 2; L <= n; L++){ //相乘矩阵个数
        for(int i = 1; i <= n; i++){ //对所有可能的矩阵相乘的情况进行遍历
            int j = i - 1 + L ;
            if(j > n) break;
            for(int k = i; k < j; k++){
                选出相乘最小的情况，由dp[i][j]记录;
                s[i][j] 记录切割位置;
            }
        }
    }
}

```

```

        printf("m[1][%d]=%d\n", n, dp[1][n]);
        traceBack(1, n);
    }

```

源代码:

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h> //包含 malloc 函数
#include <string.h> //包含 memset 函数
#define min(x, y) x < y ? x : y

/*
example :
6
30 35
35 15
15 5
5 10
10 20
20 25
*/
//expected: dp[1][6] = 15125
int dp[100][100]; //二维数组存储动态最优解
int p[100][2]; //存储对应 Matrix 的行列数
int s[10][10];

void traceBack(int i, int j){ //i -> j 的运算顺序 (回溯法实现, 树的后序遍历)
    //recursion base
    if(i == j)
        return ;
    int k = s[i][j];
    traceBack(i, k);
    traceBack(k+1, j);
    printf("A%d ... A%d    A%d ... A%d\n", i, k+1, k+1, j);
}

int main(int argc, char *argv[]) {
    int n;
    //    printf("Please input the number of maxtrix >>");
    scanf("%d", &n);

```

```

        for(int i = 1; i <= n; i++){
            for(int j = 0; j < 2; j++){
                scanf("%d", &p[i][j]);
                printf("%d ", p[i][j]);
            }
        }
        memset(dp, 2, sizeof(dp)); //utilize 异常 -> 初始化为正无穷
        for(int i = 1; i <= n; i++) //对应位置初始化!!
            dp[i][i] = 0; //最小子问题初始化

        //core code
        for(int L = 2; L <= n; L++){ //相乘矩阵个数
            for(int i = 1; i <= n; i++){ //对所有可能的矩阵相乘的情况进行
遍历
                int j = i - 1 + L ; //j -> upper_bound (dynamic)
                if(j > n) break;
                for(int k = i; k < j; k++){ // k : i -> j - 1 遍历切割
位置
                    //                printf("%d ", dp[i][k] + dp[k + 1][j] +
                    //                p[i][0] * p[k][1] * p[j][1]);
                    //-----自底向上-----
                    //                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k +
                    //                1][j] + p[i][0] * p[k][1] * p[j][1]);
                    //min 宏定义即可等效替代
                    int min = dp[i][j];
                    int tmp = dp[i][k] + dp[k + 1][j] + p[i][0] *
p[k][1] * p[j][1];

                    if(tmp < min){
                        dp[i][j] = tmp;
                        min = tmp;
                        s[i][j] = k; //i -> j 最有切割位置为 k
                    }
                }
            }
            //                putchar('\n');
        }
        //结果可视化~查表 -> 大大提高效率
        printf("%d 个矩阵连乘最优解: \n", L);
        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                printf("%d ", dp[i][j]);
            }
            putchar('\n');
        }
    }
}

```

```
//      for(int i = 1; i <= n; i++){
//          for(int j = 1; j <= n; j++){
//              printf("%d ", dp[i][j]);
//          }
//          putchar('\n');
//      }
//      for(int i = 1; i <= n; i++){
//          for(int j = 1; j <= n; j++){
//              printf("%d ", s[i][j]);
//          }
//          putchar('\n');
//      }
//      printf("m[1][6]=%d\n", n, dp[1][n]);
//      traceBack(1, n);
//  }
```

运行结果:

m[1][6]=15125

A2 ... A3	A3 ... A3
A1 ... A2	A2 ... A3
A4 ... A5	A5 ... A5
A4 ... A6	A6 ... A6
A1 ... A4	A4 ... A6

时间复杂度分析:

$O(n^3)$

空间复杂度分析:

$O(n^2)$

二、实验总结

1. 掌握了动态规划法的核心设计思想，锻炼了用空间换时间的思维（开数组记录重复计算的子问题），以后做题时，要对题目深刻分析，压榨其时间性能；

2. 掌握了建立状态方程的方法，建立了应用动态规划法解决实际问题的能力。
3. 相对其他算法，虽然空间复杂度会变高，但时间复杂度会降低很多，非常值的一件事儿。
4. 巧妙利用伪代码，将自己的思维清晰的展现在别人面前，提高共同的效率。